

1.1 Bits and Their Storage

Inside today's computers information is encoded as patterns of 0s and 1s. These digits are called **bits** (short for binary digits).

Boolean operations

Operations that manipulate true/false values are called **Boolean operations**.

The AND operation

$\begin{array}{r} 0 \\ \text{AND } 0 \\ \hline 0 \end{array}$	$\begin{array}{r} 0 \\ \text{AND } 1 \\ \hline 0 \end{array}$	$\begin{array}{r} 1 \\ \text{AND } 0 \\ \hline 0 \end{array}$	$\begin{array}{r} 1 \\ \text{AND } 1 \\ \hline 1 \end{array}$
---	---	---	---

The OR operation

$\begin{array}{r} 0 \\ \text{OR } 0 \\ \hline 0 \end{array}$	$\begin{array}{r} 0 \\ \text{OR } 1 \\ \hline 1 \end{array}$	$\begin{array}{r} 1 \\ \text{OR } 0 \\ \hline 1 \end{array}$	$\begin{array}{r} 1 \\ \text{OR } 1 \\ \hline 1 \end{array}$
--	--	--	--

The XOR operation

$\begin{array}{r} 0 \\ \text{XOR } 0 \\ \hline 0 \end{array}$	$\begin{array}{r} 0 \\ \text{XOR } 1 \\ \hline 1 \end{array}$	$\begin{array}{r} 1 \\ \text{XOR } 0 \\ \hline 1 \end{array}$	$\begin{array}{r} 1 \\ \text{XOR } 1 \\ \hline 0 \end{array}$
---	---	---	---

Gates and Flip Flops

Gate

A gate is a device that produces the output of a Boolean operation when given the input values.

Gates provide the building blocks from which computers are constructed.

hierarchical structure: gate $\rightarrow$ circuit $\rightarrow$ system

AND



Inputs	Output
0 0	0
0 1	0
1 0	0
1 1	1

OR



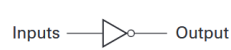
Inputs	Output
0 0	0
0 1	1
1 0	1
1 1	1

XOR



Inputs	Output
0 0	0
0 1	1
1 0	1
1 1	0

NOT



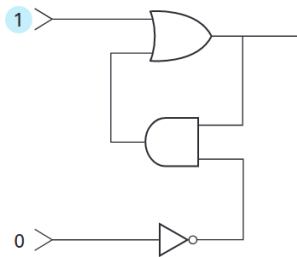
Inputs	Output
0	1
1	0

Flip-Flop

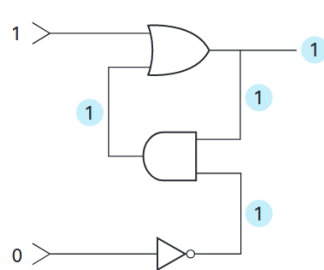
A flip-flop is a fundamental unit of computer memory.

The output remains constant until a pulse changes it. (memory)

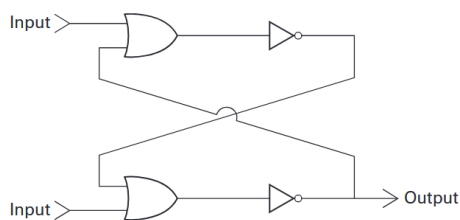
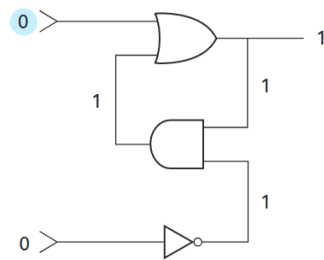
a. First, a 1 is placed on the upper input.



b. This causes the output of the OR gate to be 1 and, in turn, the output of the AND gate to be 1.



c. Finally, the 1 from the AND gate keeps the OR gate from changing after the upper input returns to 0.



the output feeds back into the circuit/feedback loop

The importance of flip-flop:

It demonstrates how devices can be constructed from gates.

A flip-flop provides an example of abstraction and the use of abstract tools.

It is one means of storing a bit within a modern computer.

VLSI (Very Large Scale Integration) chip = millions of flip-flops

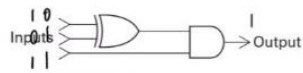
Hexadecimal Notation

A long string of bits is often called a **stream**.

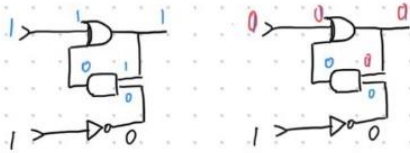
Hexadecimal notation is a shorthand representation of bit patterns in which one symbol represents four bits.

Bit pattern	Hexadecimal representation
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

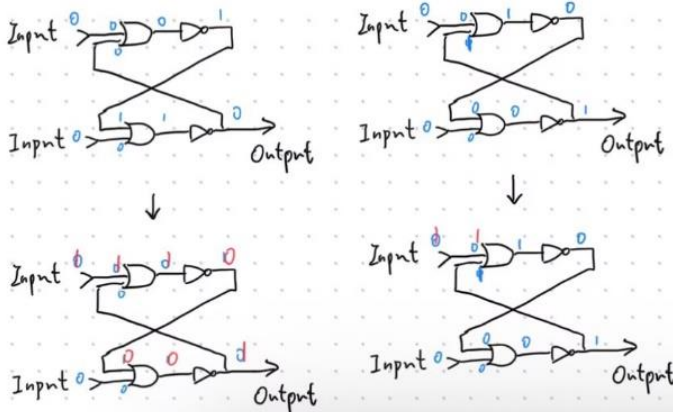
1. What input bit patterns will cause the following circuit to produce an output of 1?



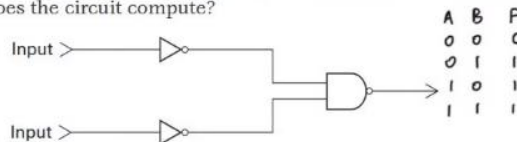
2. In the text, we claimed that placing a 1 on the lower input of the flip-flop in Figure 1.3 (while holding the upper input at 0) will force the flip-flop's output to be 0. Describe the sequence of events that occurs within the flip-flop in this case.



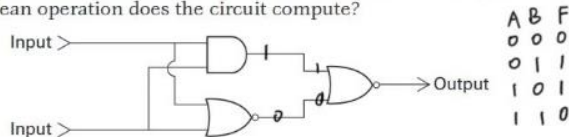
3. Assuming that both inputs to the flip-flop in Figure 1.5 begin as 0, describe the sequence of events that occurs when the upper input is temporarily set to 1.



4. a. If the output of an AND gate is passed through a NOT gate, the combination computes the Boolean operation called NAND, which has an output of 0 only when both its inputs are 1. The symbol for a NAND gate is the same as an AND gate except that it has a circle at its output. The following is a circuit containing a NAND gate. What Boolean operation does the circuit compute?



- b. If the output of an OR gate is passed through a NOT gate, the combination computes the Boolean operation called NOR that has an output of 1 only when both its inputs are 0. The symbol for a NOR gate is the same as an OR gate except that it has a circle at its output. The following is a circuit containing an AND gate and two NOR gates. What Boolean operation does the circuit compute?



5. Use hexadecimal notation to represent the following bit patterns:

- a. 0110 0101 1110 1010 6AF2 b. 1110 1000 0101 0101 0001 0111 E85517
- c. 0100 0000 48 d. 0100 0000 0000 0000 0000

6. What bit patterns are represented by the following hexadecimal patterns?

- a. 5FD97 b. 610A c. ABCD d. 0100

15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0	5 F D 9 7	0 1 0 1 1 1 1 1 1 0 1 1 0 0 1 0 1 1 1
15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0	6 1 0 A	0 1 1 0 0 0 0 1 0 0 0 0 1 0 1 0
15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0	A B C D	1 0 1 0 1 0 1 1 1 1 0 0 1 1 0 1
15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0	0 1 0 0	0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0

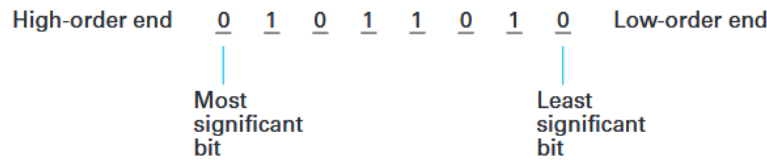
1.2 Main Memory

Main memory is a large collection of circuits capable of storing bits.

Memory Organization

Memory Cell: Main memory is divided into units called memory cells.

A string of eight bits is called a **byte**.



Each memory cell has a unique name called an **address**.

Longer bit patterns can be stored using multiple consecutive memory cells.

A **read operation** retrieves the contents of a memory cell.

A **write operation** stores a bit pattern in a memory cell.

Random Access Memory (RAM)

Main memory is called **random access memory (RAM)** because memory cells can be accessed in any order.

Dynamic RAM (DRAM) stores bits as tiny electric charges that must be periodically refreshed.

Synchronous DRAM (SDRAM) is a type of DRAM designed to improve memory access speed.

Measuring Memory Capacity

Kilobyte (KB) 1 KB = 1024 bytes

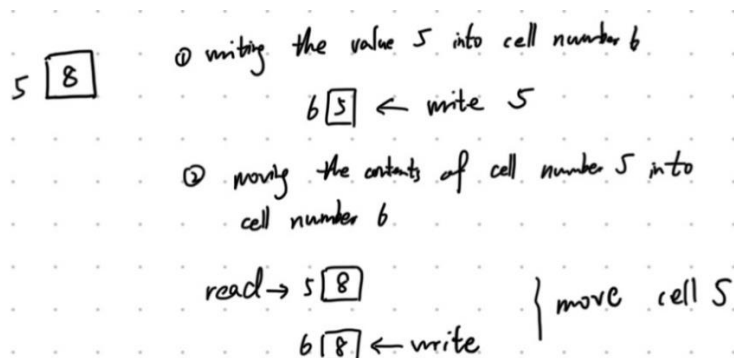
Larger Memory Units:

1 MB = 2^{20} bytes

1 GB = 2^{30} bytes

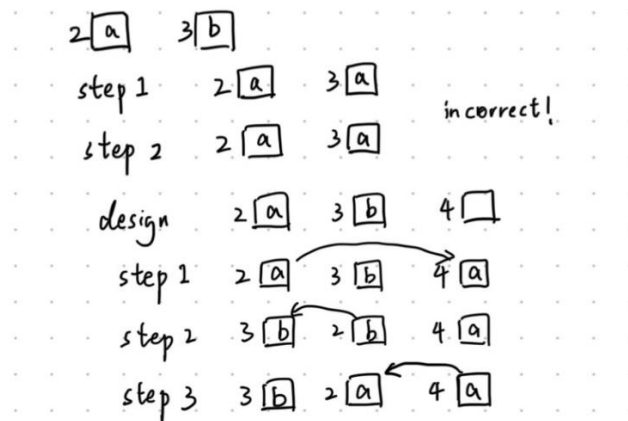
1 TB = 2^{40} bytes

1. If the memory cell whose address is 5 contains the value 8, what is the difference between writing the value 5 into cell number 6 and moving the contents of cell number 5 into cell number 6?



2. Suppose you want to interchange the values stored in memory cells 2 and 3. What is wrong with the following sequence of steps: Step 1. Move the contents of cell number

2 to cell number 3. Step 2. Move the contents of cell number 3 to cell number 2.
Design a sequence of steps that correctly interchanges the contents of these cells. If needed, you may use additional cells.



3. How many bits would be in the memory of a computer with 4KB memory?

$$4 \text{ KB} = 4 \times 2^{10} \times 8 \text{ bits} = 32768 \text{ bits}$$

↓
2¹⁰ bytes

1.3 Mass Storage

Mass storage (also called **secondary storage**) refers to additional memory devices used to store large amounts of data **outside the main memory**.

Examples: Magnetic disks/ CDs and DVDs/ Magnetic tapes/ Flash drives/ Solid-state disks (SSDs)

These devices store data permanently or semi-permanently.

Advantages: Lower volatility/ Large storage capacity/ Lower cost per unit of storage/

Removable media (in some cases)

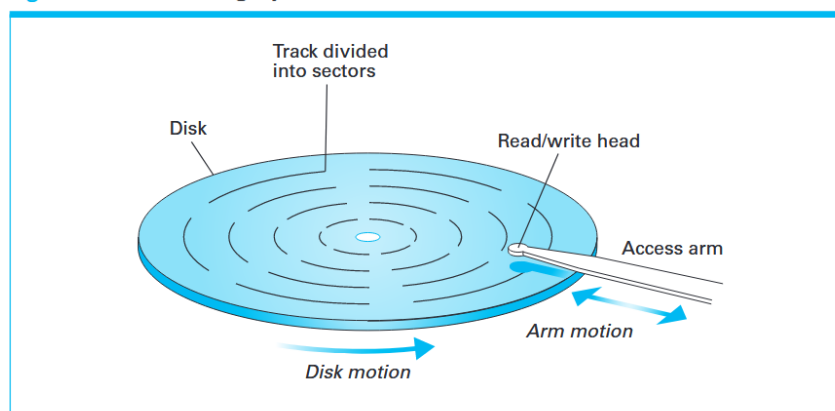
Disadvantages: Slower speed/ Mechanical failure risk

Magnetic Systems

Magnetic Disk (Hard Disk Drive, HDD)

They store data on thin spinning disks coated with magnetic material.

Figure 1.9 A disk storage system



Platter: the disk surface where data is stored

Spindle: the axis that rotates the disks

Track: A track is a circular path on the disk surface where data is stored.

Sector: Each track is divided into smaller segments called sectors. Each sector stores a fixed amount of data, typically 512 bytes to a few kilobytes.

Cylinder: A cylinder is a group of tracks located at the same radius on different platters. When the read/write heads move to a new position, a new cylinder becomes accessible.

Zoned-Bit Recording: This technique is called zoned-bit recording. Tracks are grouped into zones, and tracks in the same zone have the same number of sectors.

Disk Capacity depends on: the number of platters/ the density of tracks/ the density of sectors

Disk Performance Metrics:

1)**Seek time** is the time required to move the read/write head from one track to another.

2)**Rotational Delay (Latency)** Rotational delay or latency time is the time waiting for the desired sector to rotate under the read/write head.

3)**Access Time** Access time is the total time required to access data.

Access Time = Seek Time + Rotation Delay

4)**Transfer Rate** Transfer rate is the speed at which data can be transferred to or from the disk. It is usually measured in MB per second.

Head Crash

The read/write heads do not touch the disk surface. They float extremely close to the disk. If dust particles enter the system, the head may collide with the disk surface.

This damage is called a head crash.

Disk systems are much slower than electronic circuits because they require **mechanical motion**.

Magnetic Tape: Magnetic tape stores data on magnetic-coated plastic tape wound on reels. It has very long seek times, but it offers low cost and high capacity, making it suitable for archival backups.

Floppy Disk (软盘): A floppy disk is a portable magnetic disk enclosed in a removable cartridge.

It has low data density and slow speed, but it was once valuable because of its portability.

Optical Systems

Optical storage systems store data using light (laser technology) to read information from a reflective surface.

-Compact Disk (CD)

A compact disk (CD) is about 12 centimeters in diameter and consists of a reflective surface covered by a protective layer.

Data is recorded by creating irregularities on the reflective surface.

How Data Is Read from a CD: Data on a CD is read by a laser beam, which detects variations in the reflective surface as the disk spins.

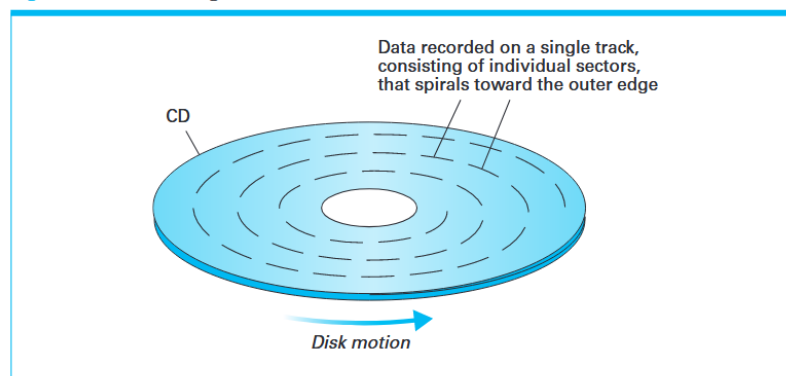
CD-DA (compact disk-digital audio) recoding format

Spiral Track: Unlike magnetic disks, data on a CD is stored on a single spiral track that starts from the center and spirals outward.

Sectors on CDs: The spiral track is divided into sectors, each containing about 2 KB of data. CDs store data at a uniform linear density, meaning the amount of data per unit length of track is constant.

CD-DA players vary the rotation speed depending on the position of the laser to maintain a constant data transfer rate.

Figure 1.10 CD storage format



-DVD (Digital Versatile Disc): DVDs use multiple semitransparent layers, allowing a laser to read data from different layers. This increases the storage capacity to several gigabytes.

-Blu-ray Disc: Blu-ray discs use a blue-violet laser, which has a shorter wavelength and allows more precise focusing. This enables Blu-ray discs to store over five times more data than DVDs.

Flash Drives

Flash memory is a type of electronic, nonvolatile 非易失存储 storage technology that stores data by trapping electrons in tiny silicon dioxide chambers(二氧化硅“腔室”).

not require mechanical motion

Limitations: Block Erasing/ Limited Lifetime

Applications: portable devices

-A **flash drive**(U 盘) is a portable flash memory device used for general data storage.

These drives may have capacities of hundreds of gigabytes.

-**SSD (Solid-State Disk 固态硬盘)** a large flash memory device designed to replace traditional hard disks

Advantages: faster access time/ resistance to vibration and physical shock/ quiet operation (no moving parts)

Disadvantage: more expensive than hard disks

Wear-Leveling Technique: SSDs use wear-leveling techniques to extend their lifetime by distributing write operations across different memory blocks.

SD Cards: SD cards are small flash memory devices commonly used in portable electronics.

SD (Secure Digital) – up to about 2 GB

SDHC (High Capacity) – up to about 32 GB

SDXC (Extended Capacity) – up to more than 1 TB

1. What is gained by increasing the rotation speed of a disk or CD?

Increasing the rotation speed reduces **rotation delay (latency)** and increases the **data transfer rate**, allowing data to be accessed and transferred more quickly.

2. When recording data on a multiple-disk storage system, should we fill a complete disk surface before starting on another surface, or should we first fill an entire cylinder before starting on another cylinder?

We first fill an entire cylinder before starting on another cylinder.

Because all tracks in a cylinder can be accessed **without moving the read/write heads**, which reduces seek time.

3. Why should the data in a reservation system that is constantly being updated be stored on a magnetic disk instead of a CD or DVD?

SSDs use wear-leveling techniques to extend their lifetime by distributing write operations across different memory blocks.

4. What factors allow CD, DVD, and Blu-ray disks all to be read by the same drive?

Modern drives use different laser wavelengths and adjustable focusing mechanisms, allowing them to read CDs, DVDs, and Blu-ray disks.

5. What advantage do flash drives have over the other mass storage systems introduced in this section?

Portable/ faster access time/ resistance to vibration and physical shock/ quiet operation (no moving parts)

6. What advantages continue to make magnetic hard disk drives competitive?

large storage capacity at a relatively low cost.

1.4 Representing Information as Bit Patterns

Representing Text

Text is represented in computers using a code that assigns a unique bit pattern to each symbol, such as letters, digits, and punctuation marks.

Character Bit Pattern

A	01000001
B	01000010

ASCII (American Standard Code for Information Interchange)

ASCII also includes **control characters**, such as line feed(换行), carriage return(回车), and tab(制表符), which control how text is displayed.

Unicode Bit Length Unicode uses up to 21 bits to represent each character.

UTF-8 Encoding UTF-8 is a Unicode encoding format that represents characters using 8, 16, 24, or 32 bits, depending on the character.

Compatibility with ASCII UTF-8 maintains compatibility with ASCII by representing the original ASCII characters using 8-bit patterns.

Text File A text file is a file containing a sequence of characters encoded using ASCII or Unicode. manipulated by utility programs called **text editors**.

Word Processor File while a word processor file also contains formatting information such as fonts and alignment. produced by **word processors** such as Microsoft's Word

ASCII (7bits)

↓

Extended ASCII (8bits)

↓

Unicode (21bits)

↓

UTF-8 (8,16,24,32bits)

Representing Numeric Values

binary notation

However, as we will shortly see, by using binary notation we can store any integer in the range from 0 to 65535 in these 16 bits.

文本 → 用 ASCII / Unicode

数字 → 用二进制整数

two's complement

8 bit 范围

-128 ~ 127

Floating point notation

3.14

0.75

4.5

Representing Images

Pixel One way to represent an image is to treat it as a collection of pixels (picture elements).

bit map Each pixel is encoded, and the entire image becomes a bit map.

Geometric Image Representation: computer-aided design (CAD)

Bit Map	Geometric Representation
Image stored as pixels	Image stored as shapes
Easy to display	Easy to scale
Loses quality when enlarged	Maintains sharpness

Representing Sound

Sound is commonly represented by sampling the amplitude of a sound wave at regular intervals.

Each sample records the amplitude value of the sound wave at a specific moment in time.

The sequence of these values represents the sound.

Sample Rate: The sample rate is the number of samples taken per second.

High-Quality Audio (CD Quality)

Stereo Sound: Stereo recordings require two channels: left channel and right channel

MIDI Encoding: Instead of recording the sound itself, MIDI records instructions for producing the sound.

Sampling Audio MIDI

Stores sound waves Stores musical instructions

Large storage Very small storage

Exact reproduction Depends on synthesizer(合成器)

Questions											
1. Here is a message encoded in ASCII using 8 bits per symbol. What does it say? (See Appendix A.) 01000111 01101111 01101101 01110000 01110101 01110100 01100101 01110110 00100000 01010011 01100011 01101001 01100101 01101110 01100011 01100101 2. In the ASCII code, what is the relationship between the codes for an uppercase letter and the same letter in lowercase? (See Appendix A.) $a = A + 32$											
二进制	十进制	十六进制	图形	二进制	十进制	十六进制	图形	二进制	十进制	十六进制	图形
0010 0000	32	20	(space)	0100 0000	64	40	@	0110 0000	96	60	`
0010 0001	33	21	!	0100 0001	65	41	A	0110 0001	97	61	a
0010 0010	34	22	"	0100 0010	66	42	B	0110 0010	98	62	b
0010 0011	35	23	#	0100 0011	67	43	C	0110 0011	99	63	c
0010 0100	36	24	\$	0100 0100	68	44	D	0110 0100	100	64	d
0010 0101	37	25	%	0100 0101	69	45	E	0110 0101	101	65	e
0010 0110	38	26	&	0100 0110	70	46	F	0110 0110	102	66	f
0010 0111	39	27	'	0100 0111	71	47	G	0110 0111	103	67	g
0010 1000	40	28	(	0100 1000	72	48	H	0110 1000	104	68	h
0010 1001	41	29	)	0100 1001	73	49	I	0110 1001	105	69	i
0010 1010	42	2A	*	0100 1010	74	4A	J	0110 1010	106	6A	j
0010 1011	43	2B	+	0100 1011	75	4B	K	0110 1011	107	6B	k
0010 1100	44	2C	,	0100 1100	76	4C	L	0110 1100	108	6C	l
0010 1101	45	2D	-	0100 1101	77	4D	M	0110 1101	109	6D	m
0010 1110	46	2E	.	0100 1110	78	4E	N	0110 1110	110	6E	n
0010 1111	47	2F	/	0100 1111	79	4F	O	0110 1111	111	6F	o
0011 0000	48	30	0	0101 0000	80	50	P	0111 0000	112	70	p
0011 0001	49	31	1	0101 0001	81	51	Q	0111 0001	113	71	q
0011 0010	50	32	2	0101 0010	82	52	R	0111 0010	114	72	r
0011 0011	51	33	3	0101 0011	83	53	S	0111 0011	115	73	s
0011 0100	52	34	4	0101 0100	84	54	T	0111 0100	116	74	t
0011 0101	53	35	5	0101 0101	85	55	U	0111 0101	117	75	u
0011 0110	54	36	6	0101 0110	86	56	V	0111 0110	118	76	v
0011 0111	55	37	7	0101 0111	87	57	W	0111 0111	119	77	w
0011 1000	56	38	8	0101 1000	88	58	X	0111 1000	120	78	x
0011 1001	57	39	9	0101 1001	89	59	Y	0111 1001	121	79	y
0011 1010	58	3A	:	0101 1010	90	5A	Z	0111 1010	122	7A	z
0011 1011	59	3B	;	0101 1011	91	5B	[	0111 1011	123	7B	{
0011 1100	60	3C	<	0101 1100	92	5C	\	0111 1100	124	7C	
0011 1101	61	3D	=	0101 1101	93	5D	]	0111 1101	125	7D	}
0011 1110	62	3E	>	0101 1110	94	5E	^	0111 1110	126	7E	~
0011 1111	63	3F	?	0101 1111	95	5F	_				

3. Encode these sentences in ASCII:

- a. "Stop!" Cheryl shouted. b. Does $2 + 3 = 5$?

4. Describe a device from everyday life that can be in either of two states, such as a flag on a flagpole that is either up or down. Assign the symbol 1 to one of the states and 0 to the other, and show how the ASCII representation for the letter b would appear when stored with such bits.

Turn on/off the light.

On=1 Off=0

B=98=01100010

5. Convert each of the following binary representations to its equivalent base 10 form:

- a. 0101 5 b. 1001 9 c. 1011 11
d. 0110 6 e. 10000 16 f. 10010 18

6. Convert each of the following base 10 representations to its equivalent binary form:

- a. 6 0110 b. 13 1101 c. 11 1011
d. 18 10010 e. 27 11011 f. 4 0100

7. What is the largest numeric value that could be represented with three bytes if each digit were encoded using one ASCII pattern per byte? What if binary notation were used?

8. An alternative to hexadecimal notation for representing bit patterns is **dotted decimal notation** in which each byte in the pattern is represented by its base 10 equivalent. In turn, these byte representations are separated by periods. For example, 12.5 represents the pattern 0000110000000101 (the byte 00001100 is represented by 12, and 00000101 is represented by 5), and the pattern 10001000000100000000111 is represented by 136.167. Represent each of the following bit patterns in dotted decimal notation.

- a. 0000111100001111 b. 001100110000000010000000
c. 0000101010100000

9. What is an advantage of representing images via geometric structures as opposed to bit maps? What about bit map techniques as opposed to geometric structures?

Bit Map Geometric Representation

Image stored as pixels Image stored as shapes

Easy to display Easy to scale

Loses quality when enlarged Maintains sharpness

10. Suppose a stereo recording of one hour of music is encoded using a sample rate of 44,100 samples per second as discussed in the text. How does the size of the encoded version compare to the storage capacity of a CD?

CD 标准: 44100 samples/sec 16 bits/sample 2 channels

$44100 \times 16 \times 2$

$= 1,411,200 \text{ bits/sec} \approx 176,400 \text{ bytes/sec}$

$176400 \times 3600 \approx 635 \text{ MB} \approx 700 \text{ MB (CD capacity)}$

Binary Notation

a. Base 10 system

3	7	5
Hundred	Ten	One

Representation: 375
Position's quantity: Hundred, Ten, One

b. Base two system

1	0	1	1
Eight	Four	Two	One

Representation: 1011
Position's quantity: Eight, Four, Two, One

Binary pattern [1 0 0 1 0 1]

1	x one	= 1
0	x two	= 0
0	x four	= 0
1	x eight	= 8
0	x sixteen	= 0
1	x thirty-two	= 32
		<u>37 Total</u>

Value of bit Position's quantity

- Step 1. Divide the value by two and record the remainder.
- Step 2. As long as the quotient obtained is not zero, continue to divide the newest quotient by two and record the remainder.
- Step 3. Now that a quotient of zero has been obtained, the binary representation of the original value consists of the remainders listed from right to left in the order they were recorded.

Diagram illustrating the conversion of the decimal number 13 to binary using the remainder method:

- 13 divided by 2 gives a quotient of 6 and a remainder of 1.
- 6 divided by 2 gives a quotient of 3 and a remainder of 0.
- 3 divided by 2 gives a quotient of 1 and a remainder of 1.
- 1 divided by 2 gives a quotient of 0 and a remainder of 1.

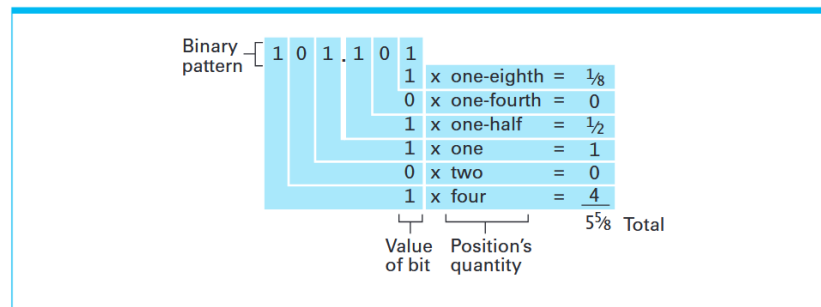
The remainders, read from bottom to top, form the binary representation: 1101.

$\frac{0}{0}$	$\frac{1}{1}$	$\frac{0}{1}$	$\frac{1}{10}$
---------------	---------------	---------------	----------------

Fractions in Binary

To extend binary notation to accommodate fractional values, we use a **radix point**(小数点) in the same role as the decimal point in decimal notation.

Figure 1.18 Decoding the binary representation 101.101



Analog(连续) versus Digital(离散)

A **digital system** represents information using **discrete digits**, typically binary digits (0 and 1).

An **analog system** represents information using a **continuous physical quantity**.

Digital systems are **more robust and less sensitive to errors**.

64 32 16 8 4 2 1 . 0.5 0.25 0.125

- Convert each of the following binary representations to its equivalent base 10 form:

a. 101010	b. 100001	c. 10111	d. 0110	e. 11111
-----------	-----------	----------	---------	----------
- Convert each of the following base 10 representations to its equivalent binary form:

a. 32	b. 64	c. 96	d. 15	e. 27
-------	-------	-------	-------	-------
- Convert each of the following binary representations to its equivalent base 10 form:

a. 11.01	b. 101.111	c. 10.1	d. 110.011	e. 0.101
----------	------------	---------	------------	----------
- Express the following values in binary notation:

a. 4 ¹ / ₂	b. 2 ³ / ₄	c. 1 ¹ / ₈	d. 5 ⁵ / ₁₆	e. 5 ⁵ / ₈
----------------------------------	----------------------------------	----------------------------------	-----------------------------------	----------------------------------
- Perform the following additions in binary notation:

a. $\begin{array}{r} 11011 \\ + 1100 \\ \hline 100111 \end{array}$	b. $\begin{array}{r} 1010.001 \\ + 1.101 \\ \hline 1011.110 \end{array}$	c. $\begin{array}{r} 11111 \\ + 0001 \\ \hline 100000 \end{array}$	d. $\begin{array}{r} 111.11 \\ + 00.01 \\ \hline 1000.00 \end{array}$
--	--	--	---

1.6 Storing Integers

Two's Complement Notation

two's complement: Two's complement notation is the most common system used in computers to represent signed integers (positive and negative numbers)

Each number is represented using a fixed number of bits, such as:

8 bits/16 bits/32 bits (very common in modern computers)

sign bit: In a two's complement system, the leftmost bit is the sign bit.

0 → non-negative number

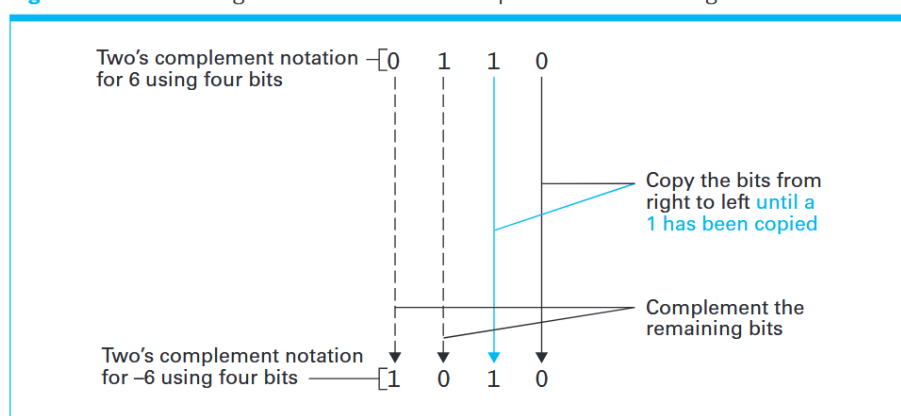
1 → negative number

Figure 1.19 Two's complement notation systems

a. Using patterns of length three		b. Using patterns of length four	
Bit pattern	Value represented	Bit pattern	Value represented
011	3	0111	7
010	2	0110	6
001	1	0101	5
000	0	0100	4
111	-1	0011	3
110	-2	0010	2
101	-3	0001	1
100	-4	0000	0
		1111	-1
		1110	-2
		1101	-3
		1100	-4
		1011	-5
		1010	-6
		1001	-7
		1000	-8

Range: $-2^{(n-1)}$ to $2^{(n-1)} - 1$

Figure 1.20 Encoding the value—6 in two's complement notation using 4 bits



Two's complement allows computers to perform **subtraction using addition circuits**, which simplifies hardware design.

Addition in Two's Complement Notation

Figure 1.21 Addition problems converted to two's complement notation

Problem in base 10		Problem in two's complement		Answer in base 10
$\begin{array}{r} 3 \\ + 2 \\ \hline \end{array}$	→	$\begin{array}{r} 0011 \\ + 0010 \\ \hline 0101 \end{array}$	→	5
$\begin{array}{r} -3 \\ + -2 \\ \hline \end{array}$	→	$\begin{array}{r} 1101 \\ + 1110 \\ \hline 1011 \end{array}$	→	-5
$\begin{array}{r} 7 \\ + -5 \\ \hline \end{array}$	→	$\begin{array}{r} 0111 \\ + 1011 \\ \hline 0010 \end{array}$	→	2

The Problem of Overflow

Overflow occurs when the result of a computation exceeds the range of values that can be represented with the available number of bits.

正 + 正 = 负 overflow

负 + 负 = 正 overflow

正 + 负 never overflow

Excess Notation

Figure 1.22 An excess eight conversion table

Bit pattern	Value represented
1111	7
1110	6
1101	5
1100	4
1011	3
1010	2
1001	1
1000	0
0111	-1
0110	-2
0101	-3
0100	-4
0011	-5
0010	-6
0001	-7
0000	-8

真实值 = 二进制值 - 8

Figure 1.23 An excess notation system using bit patterns of length three

Bit pattern	Value represented
111	3
110	2
101	1
100	0
011	-1
010	-2
001	-3
000	-4

真实值 = 二进制值 - 4

64 32 16 8 4 2 1

1. Convert each of the following two's complement representations to its equivalent base 10 form:

a. 00011 ³ b. 01111 ¹⁵ c. 00100 ⁻⁴
 d. 11010 ⁻⁶ e. 00000 ⁰ f. 10000 ⁻¹⁶

2. Convert each of the following base 10 representations to its equivalent two's complement form using patterns of 8 bits:

a. 6 0000 0110 b. -6 1111 1010 c. -12 1111 0100
 d. 13 0000 1101 e. -13 1111 0001 f. 0 0000 0000

3. Suppose the following bit patterns represent values stored in two's complement notation. Find the two's complement representation of the negative of each value:

a. 11111111 b. 01010101 c. 00000100
 d. 00000001 e. 00000000 f. 01111111

4. Suppose a machine stores numbers in two's complement notation. What are the largest and smallest numbers that can be stored if the machine uses bit patterns of the following lengths?

a. four $2^4 - 1 = 7$ b. six $2^6 - 1 = 31$ c. eight $2^8 - 1 = 127$

5. In the following problems, each bit pattern represents a value stored in two's complement notation. Find the answer to each problem in two's complement notation by performing the addition process described in the text. Then check your work by translating the problem and your answer into base 10 notation.

a. 0101 + 0010 b. 0011 + 0001 c. 0101 + 1010
 d. 1110 + 0011 e. 1010 + 1110

a. 0101
+ 0010

0111

b. 0011
+ 0001

0100

c. 0101
+ 1010

1111

d. 1110
+ 0011

10001

e. 1010
+ 1110

01000

6. Solve each of the following problems in two's complement notation, but this time watch for overflow and indicate which answers are incorrect because of this phenomenon. 8 4 2 1 ^{7 ~ -8}

a. 0100 + 0011 b. 0101 + 0110 c. 1010 + 1010
 d. 1010 + 0111 e. 0111 + 0001

a. 0100
+ 0011

0111
4 + 3 = 7

b. 0101
+ 0110

1011
5 + 6 = 11

c. 1010 ⁻⁶
+ 1010 ⁻⁶

10100
^{-6 - 6 = -12}
overflow

d. 1010 ⁻⁶
+ 0111 ⁷

10001
^{-6 + 7 = 1}
overflow

e. 0111 ⁷
+ 0001 ¹

1000
^{7 + 1 = 8}
overflow

7. Translate each of the following problems from base 10 notation into two's complement notation using bit patterns of length four, then convert each problem to an equivalent addition problem (as a machine might do), and perform the addition. Check your answers by converting them back to base 10 notation.

- a. $6 - (-1)$ b. $3 - (-2)$ c. $4 - 6$
d. $2 - (-4)$ e. $1 - 5$

a. $6 \rightarrow 0110$
 $1 \rightarrow 0001$
 $\begin{array}{r} 0110 \\ + 0001 \\ \hline 0111 \end{array} \rightarrow 7$

b. $3 \rightarrow 0011$
 $2 \rightarrow 0010$
 $\begin{array}{r} 0011 \\ + 0010 \\ \hline 0101 \end{array} \rightarrow 5$

c. $4 \rightarrow 0100$
 $6 \rightarrow 0110$
 $-6 \rightarrow 1010$
 $\begin{array}{r} 0100 \\ + 1010 \\ \hline 1110 \end{array} \rightarrow -2$

d. $2 \rightarrow 0010$
 $4 \rightarrow 0100$
 $\begin{array}{r} 0010 \\ + 0100 \\ \hline 0110 \end{array} \rightarrow 6$

e. $1 \rightarrow 0001$
 $5 \rightarrow 0101$
 $-5 \rightarrow 1011$
 $\begin{array}{r} 0001 \\ + 1011 \\ \hline 1100 \end{array} \rightarrow -4$

8. Can overflow ever occur when values are added in two's complement notation with one value positive and the other negative? Explain your answer.

No. after plus, the result is in the range of $-2^{n-1} \sim 2^{n-1} - 1$

9. Convert each of the following excess eight representations to its equivalent base 10 form without referring to the table in the text:

- a. 1110 6 b. 0111 -1 c. 1000 0
d. 0010 -6 e. 0000 -8 f. 1001 1

10. Convert each of the following base 10 representations to its equivalent excess eight form without referring to the table in the text:

- a. $5 \rightarrow 1101$ b. $-5 \rightarrow 0011$ c. $3 \rightarrow 1011$
d. $0 \rightarrow 1000$ e. $7 \rightarrow 1111$ f. $-8 \rightarrow 0000$

11. Can the value 9 be represented in excess eight notation? What about representing 6 in excess four notation? Explain your answer.

$9 + 8 = 17 > 15$ No
 $6 + 4 = 10 > 7$ No

1.7 Storing Fractions

floating-point notation

Floating-point notation is a method used to represent real numbers (fractions) in computers.

Sign Bit

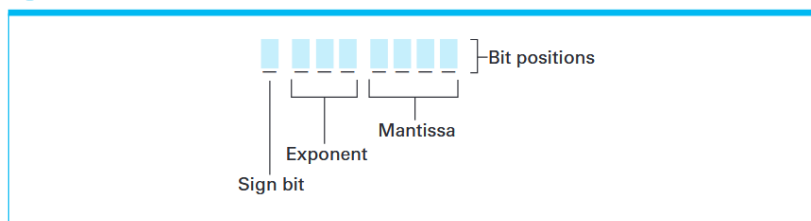
exponent field: The exponent is stored using excess notation.

Actual exponent = stored exponent $- 4$

mantissa field: The mantissa represents the significant digits of the number.

Actual exponent = stored exponent $- 4$

Figure 1.24 Floating-point notation components



Decode

01101011

0 | 110 | 1011

Mantissa 1011 $\rightarrow$.1011

110 = 6/ excess-4/ $6 - 4 = 2$

.1011 $\rightarrow$ 10.11

2.75

Encode

$11/8 = 1.375_{(10)}$

$11/8 = 1.001_{(2)}$

mantissa = 1001

compare .1001 with 1.001

Radix moved: right 1 bit

So exponent = +1

$1 + 4 = 5$ Binary: 101

sign = 0

0 101 1001

normalized form

Mantissa must start with the leftmost 1.

It prevents multiple representations of the same number.

Binary number: .011

Mantissa: 1100

NOT 0110

Special Case: Zero Zero is stored as 0 000 0000

Truncation Errors

A **truncation error** (or **round-off error**) occurs when a value cannot be stored exactly because the **mantissa field is too small**, causing some digits to be lost.

$25/8 = 10.101_2$

10.101 → mantissa = 1010

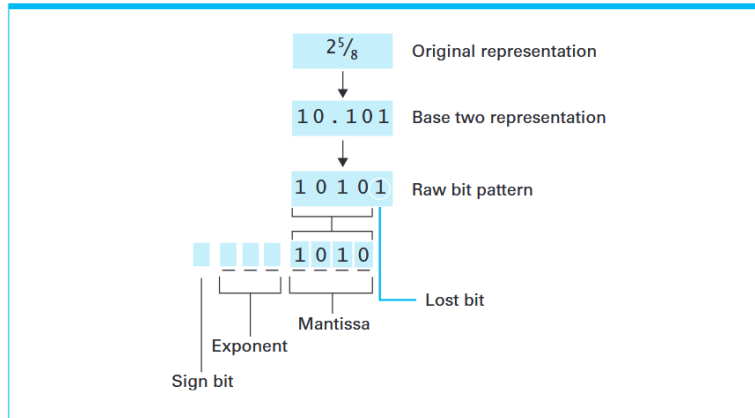
Mantissa must start from the leftmost 1:

10.101 → mantissa = 1010

But the last digit **1** cannot fit.

Thus the stored value becomes: .1010 instead of .10101

Figure 1.25 Encoding the value $2\frac{5}{8}$



Why Truncation Errors Occur

1 Limited mantissa length

If the mantissa field is too short, some digits are **cut off**.

Modern computers reduce this problem by using longer formats:

32-bit floating point

64-bit floating point

2 Non-terminating binary fractions

Some numbers **cannot be represented exactly in binary**.

Example: $1/10$

0.0001100110011...

Order of Addition Problem

Add small numbers first

Accuracy in Modern Computers

Modern floating-point systems (like IEEE 754) use:

32-bit (single precision) 64-bit (double precision)

These allow very high accuracy.

However, errors may still occur if numbers differ greatly in size.

Example: $10,000,000,000,000,000 + 1$

The result might still be: 10,000,000,000,000,000

because 1 is too small relative to the large number.

Single Precision Floating Point

Single Precision Floating Point

10^{38} (非常大) ~ 10^{-37} (非常小)

Sign | Exponent | Mantissa

1 | 8 bits | 23 bits

Double Precision Floating Point

uses 64 bits and provides a precision of 15 decimal digits.

8421 0.5 0.25 0.125 0.0625 0.03125

1. Decode the following bit patterns using the floating-point format discussed in the text:

a. 01001010 b. 01101101 c. 00111001 d. 11011100 e. 10101011

a. 01001010
0 100 1010
Mantissa 1010 $\rightarrow$.1010
100 = 4 $4-4=0$
.1010 $\rightarrow$ 0.1010
0.625

b. 01101101
0 110 1101
Mantissa 1101 $\rightarrow$.1101
110 = 6 $6-4=2$
.1101 $\rightarrow$ 11.01
3.25

c. 00111001
0 011 1001
Mantissa 1001 $\rightarrow$.1001
011 = 3 $3-4=-1$
.1001 $\rightarrow$ 0.1001
 $0.25 + 0.03125$
 $= 0.28125$

d. 11011100
1 101 1100
1100 $\rightarrow$.1100
101 = 5 $5-4=1$
.1100 $\rightarrow$ 1.100
-1.5

e. 10101011
1 010 1011
1011 $\rightarrow$.1011
010 = 2 $2-4=-2$
1011 $\rightarrow$ 0.001011
-0.171875

8 4 2 1 0.5 0.25 0.125

2. Encode the following values into the floating-point format discussed in the text. Indicate the occurrence of truncation errors.

a. $2^{3/4}$ b. $5^{1/4}$ c. $3/4$ d. $-3^{1/2}$ e. $-4^{3/8}$

$2^{3/4}$ a. $2^{3/4}$ b. $5^{1/4}$ c. $3/4$ d. $-3^{1/2}$ e. $-4^{3/8}$
10.11 101.01 0.11 start with 1 11.10 100.011
Mantissa = 1011 Mantissa = 1010 Mantissa = 001100 Mantissa = 1110 Mantissa = 1000
.1011 $\rightarrow$ 10.11 0.1010 $\rightarrow$ 0.1010 0.11 $\rightarrow$ 0.11 0.1110 $\rightarrow$ 11.10 0.1000 $\rightarrow$ 100.0
exponent +2 exponent +3 exponent 0 exponent +2 exponent +3
 $2+4=6$ 110 $3+4=7$ 111 0+4=4 100 $2+4=6$ $3+4=7$
0 110 1011 0 111 1010 0 100 1100 1 110 1110 1 111 100

3. In terms of the floating-point format discussed in the text, which of the patterns 01001001 and 00111101 represents the larger value? Describe a simple procedure for determining which of two patterns represents the larger value.

0 100 1001 0 011 1101
larger

Procedure

- 1) Compare the sign bits
- 2) If the signs are equal, compare the exponent fields
- 3) If the exponents are equal, compare the mantissas

4. When using the floating-point format discussed in the text, what is the largest value that can be represented? What is the smallest positive value that can be represented?

0 1 1 1 1 1 1 1
Mantissa 1111 $\rightarrow$.1111
 $111=7-4=3$
.1111 $\rightarrow$ 111.1
7.5

0 000 0001
Mantissa 0001 $\rightarrow$.0001
 $000=0-4=-4$
.0001 $\rightarrow$.00000001
0.00390625

1.8 Data and Programming

A **programming language** is a computer system created to allow humans to precisely express algorithms to the computer using a higher level of abstraction.

Getting Started with Python

Python is an **interpreted language**.

Hello Python

```
print('Hello, World!')
```

Hello, World!

Variables

Python allows users to name values for later use. These named storage locations are called **variables**.

```
message = 'Hello, World!'
```

```
print(message)
```

Variables can store many types of values.

```
my_integer = 5
```

```
my_floating_point = 26.2
```

```
my_Boolean = True
```

```
my_string = 'characters'
```

Hexadecimal in Python

```
my_integer = 0xFF
```

```
print(my_integer)
```

255

Python supports Unicode characters.

```
print('\u00A31000')
```

£1000

Comments in Python

```
print('\u00A31000') # prints £1000
```

Operators and Expressions

Python has built-in operators that allow values to be combined and manipulated.

```
print(3 + 4)
```

```
print(5 - 6)
```

```
print(7 * 8)
```

```
print(45 / 4)
```

```
print(2 ** 10)
```

Expression	Meaning	Result
3 + 4	addition	7
5 - 6	subtraction	-1
7 * 8	multiplication	56
45 / 4	division	11.25
2 ** 10	exponentiation	1024

Integer floor division (整除)

```
print(45 // 4)
```

11

Modulus (remainder) 取余

```
print(45 % 4)
```

```
1
```

Operators with strings

```
s = 'hello' + 'world'
```

```
helloworld
```

```
t = s * 4
```

```
helloworldhelloworldhelloworldhelloworld
```

Type Error Example

```
print('USD$' + 1000)
```

```
TypeError
```

Type conversion functions

```
int(3.9) → 3
```

```
int('25') → 25
```

```
str(1000) → '1000'
```

```
print('USD$' + str(1000)) → USD$1000
```

Currency Conversion

```
USD_to_GBP = 0.66
```

```
GBP_sign = '\u00A3'
```

```
dollars = 1000
```

```
pounds = dollars * USD_to_GBP
```

```
print('Today, $' + str(dollars))
```

```
print('converts to ' + GBP_sign + str(pounds))
```

```
converts to £660.0
```

```
integer × floating-point → floating-point
```

Debugging

Debugging is the process of **finding and fixing bugs in software**.

A **bug** is an error in a program.

Syntax Error(语法错误): Syntax errors are mistakes in the symbols or structure of the program.

Semantic Error(语义错误): Semantic errors occur when the program runs but produces the wrong result.

```
✗ total_pay = 40 + extra_hours * pay_rate
```

```
☑ total_pay = (40 + extra_hours) * pay_rate
```

Runtime Errors(运行时错误): Runtime errors occur when the program is executed.

```
print(10 / 0)
```

1. What makes Python an interpreted programming language?

Python is an interpreted programming language because its code is executed by an interpreter line by line rather than being compiled into machine code before execution.

2. Write Python statements that print the following:

a. The words “Computer Science Rocks”, followed by an exclamation point

b. The number 42

c. An approximation of the value of Pi to 4 decimal places

```
print('Computer Science Rocks!')
```

```
print(42)
```

```
print(3.1416)
```

3. Write Python statements to make the following assignments to variables:

a. The word “programmer” to a variable called, rockstar

b. The number of seconds in an hour to a variable called seconds_per_hour

c. The average temperature of the human body to a variable called bodyTemp

```
rockstar = 'programmer'
```

```
seconds_per_hour = 60 * 60
```

```
bodyTemp = 98.6°F
```

4. Write a Python statement that given an existing variable called bodyTemp in degrees Fahrenheit stores the equivalent temperature in degrees Celsius to a new variable called metricBodyTemp.

```
metricBodyTemp = (bodyTemp - 32) * 5 / 9
```

1.9 Data Compression

data compression is the process of reducing the size of data while preserving the information it contains

Generic Data Compression Techniques

Lossless Compression: Lossless compression reduces data size without losing any information.

The original data can be perfectly reconstructed after decompression.

Lossy Compression: Lossy compression reduces file size by removing some information permanently.

The original data cannot be perfectly restored.

run-length encoding (RLE) compresses data by replacing repeated characters with a count and the character

AAAAA → 5A

BBBB → 4B

AAAABBBCC → 4A3B2C

frequency-dependent encoding

Frequency-dependent encoding assigns shorter bit patterns to frequently used items and longer bit patterns to rare items.

Variable-length code (可变长度编码)

-Huffman codes

Huffman codes are frequency-dependent codes developed using the Huffman algorithm, discovered by David Huffman.

Variable-length code

relative encoding (differential encoding)

compresses data by storing differences between consecutive data units rather than entire units.

This technique is useful when consecutive data units are very similar.

Frame2 – Frame1

Frame3 – Frame2

dictionary encoding

In dictionary encoding, a message is compressed by encoding it as references to entries in a dictionary.

adaptive dictionary encoding

Adaptive dictionary encoding allows the dictionary to change during the encoding process.

New patterns discovered in the message can be added to the dictionary.

Lempel-Ziv-Welsh (LZW) encoding

The dictionary begins with basic symbols, and new patterns are added as they appear in the message.

消息: xyx xyx xyx xyx

初始字典:

1 → x

2 → y

3 → space

编码开始: xyx → 121 space → 3

得到: 1213

发现模式: xyx

加入字典: 4 → xyx

继续编码: 121343434

Compressing Images

GIF (Graphic Interchange Format)

It uses a dictionary encoding technique and limits the number of colors to 256.

Each color is stored in a palette, and each pixel is represented by a single byte referencing the palette.

JPEG (Joint Photographic Experts Group)

It is widely used for compressing color photographs.

Most digital cameras use JPEG as the default format.

1) Separate Brightness and Color

亮度 (luminance)

颜色 (chrominance)

2) Reduce Chrominance Information

JPEG reduces chrominance data by averaging color values in small pixel groups (2×2 pixels).

This reduces color information while keeping brightness intact.

3) Apply Discrete Cosine Transform (DCT)

The image is divided into 8 × 8 pixel blocks.

Each block is transformed using the Discrete Cosine Transform (DCT).

DCT converts pixel values into frequency information.

4) Remove Small Values (Lossy Step)

Values below a certain threshold are replaced with zero.

TIFF (Tagged Image File Format)

TIFF is primarily a standardized format for storing images and related metadata.

It is often used to store photographs without compression.

Format	Compression Type	Key Idea	Typical Use
GIF	Mostly lossy	palette + LZW	simple graphics / animation
JPEG	Mostly lossy	DCT + human vision limits	photographs
TIFF	usually none	storage format with metadata	high-quality images

Compressing Audio and Video

Motion Picture Experts Group (MPEG): standards for encoding and compressing audio and video.

Video is treated as a sequence of pictures (frames).

Some frames are encoded completely (**I-frames**)

Other frames are encoded using **relative encoding**

MP3 MPEG Layer 3

MP3 compresses audio by removing sounds that the human ear cannot perceive.

temporal masking (时间掩蔽)

Temporal masking occurs when a loud sound prevents the human ear from hearing softer sounds immediately afterward.

frequency masking (频率掩蔽)

Frequency masking occurs when a sound at one frequency masks softer sounds at nearby frequencies.

The goal of compressing audio and video is not only to save storage space but also to allow fast transmission over communication networks.

Transmission Speed: bits per second (bps)

Unit	Meaning
------	---------

bps	bits per second
-----	-----------------

Kbps	1000 bps
------	----------

Mbps	1,000,000 bps
------	---------------

Gbps	1,000,000,000 bps
------	-------------------

1. List four generic compression techniques.

run-length encoding (RLE)

frequency-dependent encoding

Huffman codes

relative encoding (differential encoding)

2. What would be the encoded version of the message

xyx yxxxxy xyx yxxxxy yxxxxy

if LZW compression, starting with the dictionary containing x, y, and a space (as described in the text), were used?

x -> 1

y -> 2

space -> 3

xyx -> 4

yxxxxy -> 5

121 21112 4 5 5

121321112343535

3. Why would GIF be better than JPEG when encoding color cartoons?

GIF is better for encoding color cartoons because it uses a 256-color palette, which is well suited for images with large regions of uniform color, such as cartoons.

JPEG is designed for photographic images and may introduce distortions in images with sharp color boundaries.

4. Suppose you were part of a team designing a spacecraft that will travel to other planets and send back photographs. Would it be a good idea to compress the photographs using GIF or JPEG's baseline standard to reduce the resources required to store and transmit the images?

No. It would not be a good idea because **GIF and JPEG baseline are lossy compression methods**, which may cause the loss of important scientific information.

The photographs should be stored using **lossless compression** or without compression.

5. What characteristic of the human eye does JPEG's baseline standard exploit?

亮度 (luminance) brightness

颜色 (chrominance) color

The human eye is more sensitive to brightness than to color changes.

6. What characteristic of the human ear does MP3 exploit?

Temporal masking frequency masking

7. Identify a troubling phenomenon that is common when encoding numeric information, images, and sound as bit patterns.

A troubling phenomenon is the **loss of precision**, which occurs when information such as numbers, images, or sound is approximated when represented as bit patterns.

1.10 Communication Errors

When data is stored or transmitted, there is always a risk that the bit pattern retrieved is not identical to the original.

Parity Bits

parity bit

A parity bit (奇偶校验位) is an extra bit added to a data pattern to help detect errors during storage or transmission.

odd parity (奇校验)

In an odd parity system, the total number of 1s in each pattern is always odd.

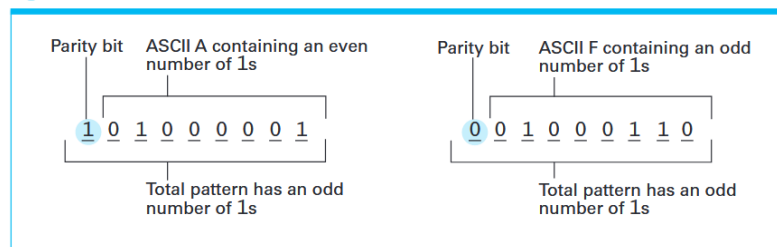
If a pattern with an even number of 1s is detected, an error has occurred.

even parity (偶校验)

In an even parity system, the total number of 1s is always even.

An error is indicated if the number of 1s becomes odd.

Figure 1.26 The ASCII codes for the letters A and F adjusted for odd parity



Modern computer memory often uses parity bits.

Although memory appears to store 8 bits, it actually stores 9 bits (8 data bits + 1 parity bit).

Parity bits can detect only an odd number of errors.

They fail to detect errors if an even number of bits are corrupted.

Checksum

A checksum is a group of parity bits, each responsible for checking different subsets of the data.

checksums and cyclic redundancy checks (CRC)

Error Correcting Codes

error-correcting codes

Error-correcting codes are encoding techniques that allow not only the detection of errors but also the correction of errors in transmitted or stored data.

Hamming distance

The Hamming distance between two bit patterns is the number of bit positions in which they differ.

If the minimum Hamming distance is 3, the code can detect up to 2-bit errors and correct 1-bit errors.

Figure 1.27 An error-correcting code

Symbol	Code
A	000000
B	001111
C	010011
D	011100
E	100110
F	101001
G	110101
H	111010

Figure 1.28 Decoding the pattern 010100 using the code in Figure 1.27

Character	Code	Pattern received	Distance between received pattern and code	
A	0 0 0 0 0 0	0 1 0 1 0 0	2	
B	0 0 1 1 1 1	0 1 0 1 0 0	4	
C	0 1 0 0 1 1	0 1 0 1 0 0	3	
D	0 1 1 1 0 0	0 1 0 1 0 0	1	Smallest distance
E	1 0 0 1 1 0	0 1 0 1 0 0	3	
F	1 0 1 0 0 1	0 1 0 1 0 0	5	
G	1 1 0 1 0 1	0 1 0 1 0 0	2	
H	1 1 1 0 1 0	0 1 0 1 0 0	4	

1. The following bytes were originally encoded using odd parity. In which of them do you know that an error has occurred?

a. 100101101 b. 100000001 c. 000000000 d. 111000000 e. 011111111

b c e

2. Could errors have occurred in a byte from question 1 without your knowing it?

Explain your answer.

Yes. They fail to detect errors if an even number of bits are corrupted.

3. How would your answers to questions 1 and 2 change if you were told that even parity had been used instead of odd?

a d

Yes. They fail to detect errors if an even number of bits are corrupted.

4. Encode these sentences in ASCII using odd parity by adding a parity bit at the high-order end of each character code:

a. "Stop!" Cheryl shouted.

11010011 11110100 11101111 01110000 10100001 00100000
01000011 01101000 11100101 11110010 01111001 11101100
00100000 01110011 01101000 11101111 01110101 11110100
11100101 01100100 10101110

b. Does 2 + 3 = 5?

11000100 11101111 11100101 01110011 00100000
00110010 00100000 10101011 00100000 10110011
00100000 00111101 00100000 10110101 10111111

5. Using the error-correcting code presented in Figure 1.27, decode the following messages:

a. 001111 100100 001100

b. 010001 000000 001011

c. 011010 110110 100000 011100

a. 001111 100100 001100

Symbol	Code	100100	001100
A	001100	2	2
B	000000	4	2
C	001111	5	4
D	010011	3	5
E	011100	3	1
F	100110	1	3
G	101001	2	3
H	110101	4	4

b. 010001 000000 001011

Symbol	Code	010001	001011
A	001011	2	3
B	000000	4	1
C	010011	1	2
D	011100	2	4
E	100110	5	4
F	101001	3	2
G	110101	2	5
H	111010	4	3

c. 011010 110110 100000 011100

Symbol	Code	011010	110110	100000	011100
A	000000	3	4	5	4
B	001111	3	4	5	4
C	010011	2	3	4	4
D	011100	2	3	4	4
E	100110	4	1	2	2
F	101001	4	5	2	3
G	110101	5	2	3	3
H	111010	1	2	3	3

距离为 0 的也要查一下表!

6. Construct a code for the characters A, B, C, and D using bit patterns of length five so that the Hamming distance between any two patterns is at least three.

A = 00000

B = 11100

C = 00111

D = 11011